

Trabalho 4 - Paralelização por MPI

Matheus Berbel Barusso

Repositório completo do trabalho, com arquivos para replicação, disponível em: <https://github.com/MatheusBarusso/sistemas-distribuidos>

I. APLICAÇÃO ESCOLHIDA

O Método de Monte Carlo remete a uma classe de métodos estatísticos que utilizam aleatoriedade para resolver problemas determinísticos, isso é, ao invés de calcular uma solução exata (e complexa) diretamente é possível obter uma aproximação por meio de milhões de simulações aleatórias. Com base nesse método a aplicação escolhida para paralelização foi a aproximação do valor de π por meio do Método de Monte Carlo. Para isso utilizamos:

- Um círculo inscrito em um quadrado de lado $2r$;
- Valores gerados aleatoriamente.

Com isso temos que:

- A área do quadrado $A_{sqr} = (2r)^2 = 4r^2$;
- A área do círculo é $A_{cir} = \pi r^2$;
- A razão entre a área do círculo e a área do quadrado é:

$$\frac{A_{cir}}{A_{sqr}} = \frac{\pi r^2}{4r^2} = \frac{\pi}{4}$$

Se um número muito grande de pontos aleatório (x, y) for gerado dentro desse quadrado e testarmos quantos caem dentro do círculo por meio da equação da circunferência $x^2 + y^2 \leq r^2$ teremos:

$$\frac{\text{Pontos dentro do círculo}}{\text{Pontos totais}} = \frac{\pi}{4}$$

E portanto:

$$\pi \approx 4 \times \frac{\text{Pontos dentro do círculo}}{\text{Pontos totais}}$$

II. IMPLEMENTAÇÃO

Foram utilizadas as bibliotecas:

- **MPI** para paralelização;
- **random** para geração de número aleatório;
- **time** para cronômetro;
- **sys** para utilização de *flags* de inicialização.

Quando iniciado no modo paralelo o programa segue o seguinte *pipeline*:

- 1) `comm = MPI.COMM_WORLD` represente processos, `rank = comm.Get_rank()` e `size = comm.Get_size()` para id e quantidade de processos, respectivamente;

Matheus Berbel Barusso, graduando em Engenharia de Computação da Universidade Tecnológica Federal do Paraná - UTFPR - Rua Cristo Rei 19, Vila Becker, Toledo-PR, CEP 85902-490 Brasil e-mail: (barusso@alunos.utfpr.edu.br).

- 2) Seed de aleatoriedade única para cada processo é criada somando valor pré-definido com o número do processo obtido em `rank`;
- 3) Quantidade de pontos a serem calculados por `rank` é calculado e variável para contagem de cada `rank` é iniciada;
- 4) `comm.Barrier()` é utilizado para garantir início sincronizado, o processo só continua para todos os `ranks` quando todos chegam neste ponto.
- 5) `time.perf_counter()` inicia o cronômetro;
- 6) Laço é iniciado e rodará em relação ao número de pontos a serem calculados por cada `rank`. No laço um valor de x e y é gerado aleatoriamente e por meio da equação da circunferência é checado se o ponto está dentro do círculo. Caso esteja o valor de pontos dentro é incrementado em 1;
- 7) Quando laço é finalizado `comm.Barrier()` faz com que todos os processos tenham finalizado antes de continuar;
- 8) Por meio de `comm.reduce(contador_local, op=MPI.SUM, root=0)` todos os valores somados individualmente por cada processo são somados (`op=MPI.SUM`) em uma variável `contador_global` que o apenas o `rank 0` (`root=0`) tem acesso;
- 9) `time.perf_counter()` finaliza o cronômetro e tempo gasto é calculado;
- 10) Apenas o processo 0 realiza o cálculo aproximado de π (pois só ele tem acesso a variável `contador_global`);
- 11) Processo 0 imprime na tela o número de processos, número total de pontos, número total de pontos dentro do círculo, valor aproximado de π e tempo gasto.

Quando o programa é executado em modo Serial a sua lógica matemática é a mesma, com as diferenças:

- Nada relacionado a paralelização é feito;
- Apenas um contador é utilizado;
- Apenas valor pré-definido de seed é utilizado;

III. EXECUÇÃO

Para a execução do programa duas flags são utilizadas:

- **serial** ou **paralelo**: Definem o modo de execução do programa;
- **TOTAL_PONTOS**: Após o modo de execução um valor para o número de total de pontos deve ser inserido.

Porém, além da flag de modo, os parâmetros de execução mudam caso o processamento seja Serial ou Paralelo:

- 1) **Execução Serial**: Em um terminal, execute:

```
python3 main.py serial NUMERO_PONTOS
```

Exemplo Serial para 100.000.000 pontos:

```
python3 main.py serial 100000000
```

2) **Execução Paralela:** Em um terminal, execute:

```
mpirun -np NUMERO_PARALELIZAO python3 ...  
main.py paralelo NUMERO_PONTOS
```

Exemplo Paralelo separado em 8 processos para 100.000.000 pontos:

```
mpirun -np 8 python3 main.py paralelo ...  
100000000
```

3) **Execução Paralela utilizando threads lógicas:** O MPI Python utiliza por padrão o número de núcleos físicos do processador e pode acusar erro quando a flag de processos é maior que o número de cores. Para "contornar" isso por meio threads lógicas em um terminal, execute:

```
mpirun --use-hwthread-cpus -np ...  
NUMERO_PARALELIZAO python3 ...  
main.py paralelo NUMERO_PONTOS
```

Exemplo Paralelo separado em 16 processos para 100.000.000 pontos:

```
mpirun --use-hwthread-cpus -np 16 ...  
python3 main.py paralelo 100000000
```

IV. RESULTADOS OBTIDOS

Para a análise da diferença de tempo utilizada testes com diferentes parâmetros serão realizados, sendo eles:

- Modo Serial;
- Modo Paralelo com 2, 4, 8, 12 e 16 processos;

E para cada um desses modos, será realizado o cálculo com 100 milhões de pontos totais e 1 bilhão de pontos totais.

TABLE I
100 MILHÕES DE PONTOS TOTAIS

Modo	Tempo Utilizado (s)	Valor de π
Serial	21,16	3,14165008
Paralelo com 2 Processos	11,54	3,14164521
Paralelo com 4 Processos	6,62	3,14162808
Paralelo com 8 Processos	4,34	3,14160508
Paralelo com 12 Processos	3,69	3,14145976
Paralelo com 16 Processos	3,72	3,14148092

TABLE II
1 BILHÃO DE PONTOS TOTAIS

Modo	Tempo Utilizado (s)	Valor de π
Serial	246,89	3,141645352
Paralelo com 2 Processos	116,63	3,141612244
Paralelo com 4 Processos	68,32	3,141601628
Paralelo com 8 Processos	43,46	3,141598882
Paralelo com 12 Processos	38,25	3,141529192
Paralelo com 16 Processos	40,57	3,141540052

V. ANÁLISE DE RESULTADOS

Para obter um valor numérico que represente a diferença nos resultados obtidos utilizaremos o *Speedup*, que é calculado pela razão entre o tempo Serial e o tempo Paralelo para cada um dos casos:

TABLE III
SPEEDUP EM 100 MILHÕES DE PONTOS TOTAIS

Modo	Speedup
Serial	1
Paralelo com 2 Processos	1.83
Paralelo com 4 Processos	3.19
Paralelo com 8 Processos	4.88
Paralelo com 12 Processos	5.74
Paralelo com 16 Processos	5.69

TABLE IV
SPEEDUP EM 1 BILHÃO DE PONTOS TOTAIS

Modo	Speedup
Serial	1
Paralelo com 2 Processos	2.11
Paralelo com 4 Processos	3.61
Paralelo com 8 Processos	5.68
Paralelo com 12 Processos	6.45
Paralelo com 16 Processos	6.09

Os resultados de *Speedup* mostram que a escalabilidade do algoritmo paralelizado se mantém consistente até 8 a 12 processos, com um crescimento quase linear. É notável também que quando 16 processos são utilizados se inicia uma saturação, isso provavelmente é causado pelo fato que o processador utilizado para os testes possui 8 núcleos físicos e 16 threads lógicas, quando esse valor é ultrapassado uma "disputa" pela CPU começa e threads podem compartilhar execução. Ao analisarmos a relação de eficiência, calculada pela razão entre *Speedup* e número de processos adicionais tempos que:

TABLE V
EFICIÊNCIA EM 100 MILHÕES DE PONTOS TOTAIS

Modo	Speedup
Paralelo com 2 Processos	91,5%
Paralelo com 4 Processos	79,7%
Paralelo com 8 Processos	61,0%
Paralelo com 12 Processos	47,8%
Paralelo com 16 Processos	35,6%

TABLE VI
EFICIÊNCIA EM 1 BILHÃO DE PONTOS TOTAIS

Modo	Speedup
Paralelo com 2 Processos	105,5%
Paralelo com 4 Processos	90,2%
Paralelo com 8 Processos	71%
Paralelo com 12 Processos	53,8%
Paralelo com 16 Processos	38,1%

A eficiência paralela apresentou resultados "inferiores" conforme o número de processos aumenta demais. Para 2 e 4

processos a eficiência é próxima de 100%, com paralelização em 2 processos para 1 bilhão de pontos totais apresentando um *Speedup superlinear*, provavelmente por conta de cache melhor aproveitado e menor pressão de memória. Quando o número de processos chega em 16 a eficiência demonstra uma queda considerável, mostrando que o ganho de tempo apresentado em relação ao número de processos paralelos não possui um crescimento indefinido.

Em relação ao valor de π encontrado, e considerando o valor real de $\pi \approx 3.141592653589793$ vemos que o resultado mais preciso para 100 milhões de pontos foi com 8 processos paralelos, apresentando um erro de aproximadamente +0.000012; para 1 bilhão de pontos o resultado mais preciso também foi quando paralelizado em 8 processos, com erro de aproximadamente +0.000006. Porém, é necessário clarificar que encontrar um valor mais próximo de pi tem relação com um maior valor para o número total de pontos (Lei dos Grandes Números) e não para o número de processos utilizados, explicitando que utilizar MPI para paralelização de um Método de Monte Carlo pode acelerar o cálculo, porém não melhora a qualidade e precisão do resultado. Os resultados demonstrarem valores diferentes para π é proveniente dos números gerados aleatoriamente em subgrupos e agregados depois.